Unifying Search Paradigms using OptDSL

Michael Bögl

October 29, 2026

Abstract

In practical combinatorial optimization, solver-paradigm decisions are frequently made at an early project stage. Such early commitments constrain subsequent modeling choices and can render later paradigm changes technically and organizationally costly. This report addresses that limitation through a workflow that enables paradigm switching over the full project lifecycle. The workflow is based on `OptDSL` as a shared problem representation, leveraging its existing MiniZinc translation and extending the same representation with a Large Neighborhood Search (LNS) backend. Beyond switching between the exact and the heuristic paradigm, the same model-centric representation provides the substrate for automated operator development: the construction, destroy, and repair operators that drive the LNS search can themselves be generated from the `OptDSL` model rather than being written by hand. We detail the LNS design, including operator profiles, initialization strategies, and restart mechanisms, and evaluate the approach on benchmark instances from multiple problem classes. Under fixed iteration and time budgets, we compare LNS-based heuristic search with a MiniZinc-based exact baseline on identical `OptDSL` instances, and we further demonstrate practical applicability on a real-world use case within an extensible, model-driven optimization workflow.

1 Introduction

Combinatorial optimization practitioners routinely face a tension between solution quality, computation time, and implementation effort. In many real projects, model requirements evolve over time, while the decision for a solver paradigm is made early and is difficult to revise later. This creates a methodological lock-in that reduces both technical flexibility and practical efficiency.

Exact methods provide strong guarantees, most notably provable optimality and bounds on solution quality. These properties are essential when results must be validated formally. However, exact methods also exhibit well-known drawbacks. Their computational cost can grow prohibitively on large or highly constrained instances, so that no acceptable solution may be available within practical time limits. Their performance is sensitive to formulation quality, solver configuration, and available hardware, which makes results difficult to predict in advance. Moreover, tight integration with a specific solver technology increases the engineering cost of model changes and rapid iteration.

Heuristic and metaheuristic methods occupy the opposite end of this trade-off. They typically deliver good solutions quickly, scale to large instances, and can exploit domain structure through

tailored search strategies. This makes them attractive under strict runtime limits or in settings with frequent re-optimization. Their drawbacks mirror their strengths: they provide no global optimality guarantees and usually no quality bounds, so the gap to the optimum remains unknown. Their performance depends strongly on design choices such as operators, initialization, restart policies, and parameter settings, and a configuration that works well for one problem class may transfer poorly to another. Consequently, substantial development and tuning effort is often required before a heuristic becomes reliable in practice.

Neither paradigm dominates the other; their strengths and weaknesses are largely complementary. Which paradigm is preferable typically changes over the course of a project, for example when instance sizes grow, when time budgets shrink, or when formal quality guarantees become necessary. This observation motivates a unifying approach in which the problem is modeled once and the solution paradigm remains exchangeable.

`OptDSL`, a Python-inspired domain-specific language for describing optimization problems, provides such a unifying representation. A single `OptDSL` model can be dispatched to conceptually different backends: an exact backend, currently realized through a translation to the constraint modeling language MiniZinc and its portfolio of solvers, and a heuristic backend, currently realized through a Large Neighborhood Search (LNS) framework that operates directly on `OptDSL` decision variables. Because both backends consume identical problem descriptions, users can defer the paradigm decision, switch paradigms as project requirements evolve, and compare both paradigms under a common protocol without re-modeling: the exact side translates the model into the MiniZinc representation, and the heuristic side imports it and operates directly on the `OptDSL` decision variables inside the LNS framework. Beyond this dispatch view, the architecture admits a forward-looking perspective, illustrated in Figure 1: a solution that the LNS backend writes back in `OptDSL` format can be re-injected into the exact pipeline, where the MiniZinc representation and its solvers are used to validate and improve the solution and to derive quality bounds. This closes the loop between heuristic and formal optimization within one consistent, model-centric workflow. In the figure, the dashed line together with the gray-shaded area marks this feedback loop (writing a solution back in `OptDSL` format and passing it to the exact backend for validation and improvement); this step is performed outside the scope of the present technical report, which provides the component optimizers but not yet an automated orchestration of the loop. The blue box, by contrast, denotes the LNS-based heuristic part; it is the component that this report evaluates, and its full integration into the feedback loop is intended to be automated in subsequent work.

This report focuses on the design and evaluation of the LNS backend within this unified architecture.

We describe the implemented operator profiles, initialization strategies, and restart mechanisms, and evaluate the resulting search behavior on benchmark instances from multiple problem classes against the MiniZinc-based exact backend on identical `OptDSL` instances. The objective is twofold: to study how far adaptive LNS design can close the quality gap under fixed compute budgets, and to demonstrate the practical value of a model-centric workflow in which the solver strategy can be revised throughout the project lifecycle.

The shared representation also unlocks a more ambitious direction: fully automated operator

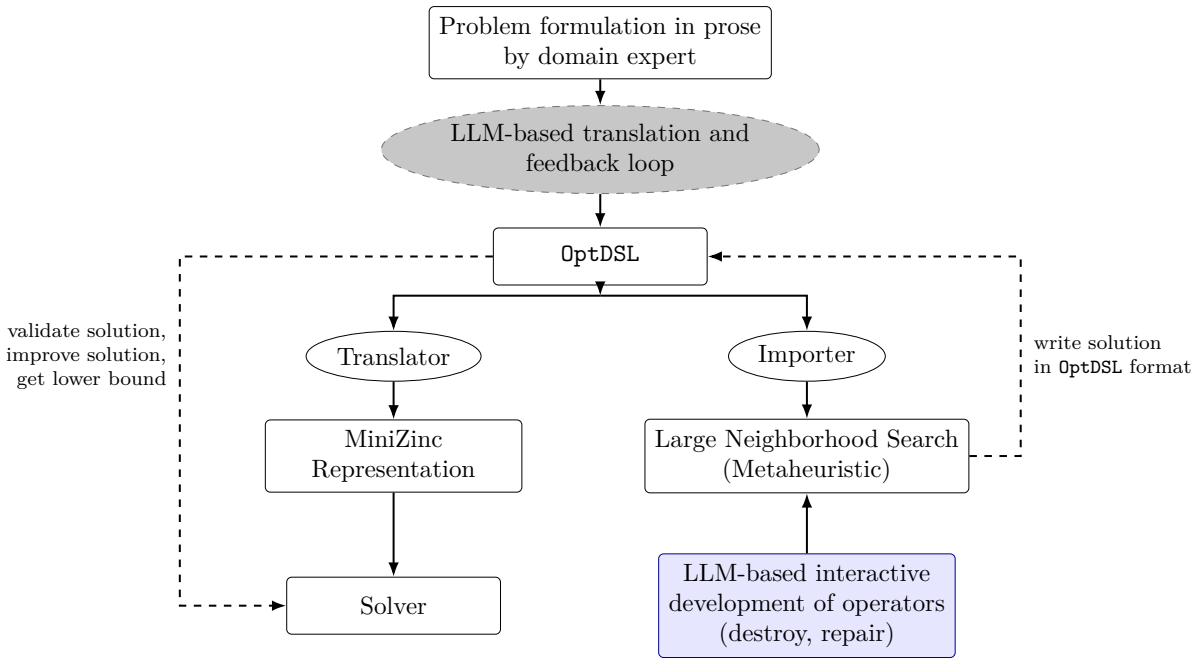


Figure 1: Perspective of the unifying workflow. The dashed line and the gray-shaded area mark the loop in which a solution is written back in `OptDSL` format and passed to the exact backend for validation and improvement; this step is performed outside the scope of the present technical report. The blue box denotes the LNS-based heuristic part, which is intended to be automated in subsequent work.

development. Because the LNS backend consumes the `OptDSL` model directly, the ingredients of the search—the constructive initializer, and the destroy and repair operators—do not have to be designed by hand. Instead, they can be generated from the model itself, validated, and iteratively improved within the unifying workflow. This extends the idea of *autoformulation* [4]—the LLM-based generation of an `OptDSL` formulation from a problem description in prose—from the model to the search: the same model-centric setting is used to also produce the search machinery that operates on the model. In this report, the operator profiles are still co-developed by a human with LLM assistance to isolate and study the behavior of the LNS backend itself; the fully automated development of these operators is the subject of ongoing work and a central next step.

2 Related Work

Research on combinatorial optimization has increasingly moved from a strict exact-versus-heuristic dichotomy toward hybrid algorithms that combine the strengths of both paradigms. In exact optimization, rich modeling languages and solver portfolios have improved accessibility and reproducibility, with MiniZinc becoming a standard interface for expressing models independently of the backend solver [17, 24]. In parallel, neighborhood-based metaheuristics such as Large Neighborhood Search (LNS) with its destroy-and-repair paradigm are well suited to large-scale, real-world combinatorial optimization problems with complex constraints, where parts of a solution are iteratively removed and rebuilt in a feasibility-preserving way [19–22].

An important algorithmic line that narrows the quality-runtime gap is the use of exact neighborhoods or exact reasoning inside heuristic search loops. In mixed-integer optimization, local branching and relaxation-induced neighborhoods are representative bridge mechanisms: they constrain search to a promising region that can still be solved (approximately or exactly) by mathematical programming machinery [7, 9]. The same principle appears in CP-LNS workflows, where exact propagation and constraint reasoning guide neighborhood repair, while the outer search remains heuristic [19, 20, 22]. Conceptually, these methods trade global proof guarantees for faster high-quality improvement, but recover part of exact-method robustness through bounded or solver-driven subproblems.

A prerequisite for such cross-paradigm studies is a shared problem representation that both exact and heuristic solvers can consume. `OptDSL` addresses this need: it is a Python-inspired domain-specific language for formulating optimization problems, and a translation from `OptDSL` to MiniZinc has been described and published [8], enabling `OptDSL` instances to be solved by any MiniZinc-compatible backend solver. The present report builds directly on that work. Rather than proposing a new modeling language or a new translation pipeline, it uses the existing `OptDSL` representation and MiniZinc translation as a foundation and contributes an experimental study that runs LNS-based heuristic search and MiniZinc-based exact solving on the same `OptDSL` instances. This enables an apples-to-apples comparison of how much algorithmic gap can be closed through shared instance engineering and adaptive neighborhood design across multiple problem classes, within the bridge-algorithm tradition described above.

3 Metaheuristic Search Framework

3.1 OptDSL Representation

All benchmark instances are provided as standalone `OptDSL` models in which data, decision variables, constraints, and the objective form a single self-contained file. Syntactically, an `OptDSL` model is a Python-inspired DSL [8]: scalar and array data are declared as typed constants (e.g. `DSList(N, DSInt())` for integer arrays), decision variables are declared with an explicit finite domain, feasibility constraints are written as `assert` statements, and the objective is declared as a final `minimize(objective)` statement. Because both backends consume exactly this file—the exact backend through the translation to MiniZinc [8], the LNS backend directly on the declared decision variables—no re-modeling is required when switching between them. We now summarize the model structure for the four benchmark classes; the first three rely on linear arithmetic over bounded integer domains, whereas woodcutting is expressed as a pipeline of record-transforming machines.

Two-dimensional bin packing (2DBP). The model operates on three finite-domain decision-variable arrays (`assignments`, `x_positions`, `y_positions`): every item i is assigned to a box b and placed at a lower-left corner (x_i, y_i). Feasibility is enforced by two constraint blocks: `all_assigned` ensures that each item receives exactly one box, and `no_overlap` first requires that each item fits within the dimensions of its selected box and then requires, for any two items in the same box, that they do not overlap in either the horizontal or the vertical direction. The objective counts the number of distinct boxes in use and is minimized.

Capacitated vehicle routing (CVRP). Two semantically equivalent encodings are used, which illustrates that `OptDSL` permits representation choices tailored to the backend. The *arc-based* encoding declares a flat binary array `arcs[v,i,j]` with index $v \cdot n^2 + i \cdot n + j$, where v indexes the vehicle, i the origin node, j the destination node, and n the number of nodes including the depot; entries are one exactly when vehicle v travels from i to j. Together with the arc array, the model declares a binary `vehicle_used` indicator and a variable u, which is used for a Miller–Tucker–Zemlin formulation. Constraints assert that every customer has in-degree and out-degree one (`all_customers_visited`), that every used vehicle starts and ends at the depot (`depot_start_end`), that flow is conserved at every customer (`flow_conservation`), that route demands respect the vehicle capacity (`capacity_constraints`), and that subtours are excluded (`subtour_elimination`); the objective sums the traveled distance. This encoding is used for the benchmark families distributed through the CVRPLIB [27] and is compatible with the MiniZinc translation. For the LNS backend on large instances we additionally use a *giant-tour* encoding with a permutation `tour` over customers and a binary `cut_after` array that marks route breaks at the depot; constraints here are occurrence, route count, and a cumulative capacity bound, and the objective inserts the depot detour at each cut. This second encoding is intentionally LNS-oriented (it operates on the permutation space of the route) and is not part of the MiniZinc baseline.

Job shop scheduling (JSSP). The model declares finite-domain completion-time variables `completion[j][op]` for every operation, bounded by `MAX_TIME`. Two constraint blocks encode feasibility: `precedence_constraint` imposes the job-internal operation order using the per-job `MACHINE_SEQUENCE` and `PROCESSING_TIMES`, and `no_overlap_constraint` asserts for every pair of operations sharing a machine that one finishes before the other starts. The objective is the makespan, i.e. the maximum completion time over all jobs. The exact backend solves the completion-time formulation directly. The LNS repair operator instead works on the relative order of operations per machine and derives an active schedule from that order; because an optimal schedule is always recoverable from some machine order [2], searching this order space does not sacrifice any optimum.

Woodcutting (industrial use case). The three classes above are flat constraint sets over decision variables. The woodcutting problem, taken from an industrial sawing line, shows that `OptDSL` also represents substantially more complex models. Instead of a single constraint block, the model is a *pipeline* of machines that exchange whole record lists at module level: an optional board-reordering machine, cutting, filtering, piece reordering, and a checking machine. The decision variables are the cut positions of every board (`cuts_list_list`), a keep/discard bit per piece (`keep_decisions`), the piece-reordering bits (`swapping_decisions`), and, in the board-reordering variant, the board-reordering bits (`swapping_decisions_boards`). Piece reordering models a single-piece buffer, and board reordering applies adjacent swaps from left to right. The checking machine then imposes the beam constraints on the stream of kept pieces: layer lengths, seam positions relative to the usable part of the beam and to static forbidden zones, minimum seam distances between consecutive layers, and an exact number of completed layers. The objective is the total length of good material that is discarded. This class is therefore not a toy formulation: it combines record-valued data, list-transforming predicates, and side conditions

that depend on the order of the produced pieces, all expressed in the same `OptDSL` syntax that the exact and the heuristic backend consume. It is evaluated separately in Section 5.2.

3.2 LNS Configuration

The LNS backend instantiates the classical destroy-and-repair scheme of Shaw [22] and combines it with the adaptive operator-selection and weight-update mechanism of ALNS [19–21]. It was not the goal to design a general LNS algorithm that works for every problem formulation in `OptDSL`; instead, the backend reads the `OptDSL` model and relies on operators that are customized and specialized for the specific problem at hand. Operator profiles are realized per problem class; in particular, the job-shop repair operator constructs active schedules in the spirit of Applegate and Cook [2]. For each class, a complete profile consists of three complementary operators: a *construction* operator that produces a first feasible assignment from the `OptDSL` model (an initializer), a *destroy* operator that removes part of a current solution, and a *repair* operator that restores a feasible complete assignment. The profiles used in the computational study were co-developed with the assistance of a large language model.

The representation-based workflow, however, does not require manually developed operators: because the LNS backend reads the `OptDSL` model directly, the construction, destroy, and repair operators can be generated automatically from the model, validated against the shared semantics, and iteratively improved. We call this *autodevelopment* of the search machinery: a large language model reads the `OptDSL` model—its data, decision variables, constraints, and objective—and writes the construction, destroy, and repair operators as plain, self-contained Python that registers itself with the LNS engine. Because the generated code uses the same registration interface as a hand-written profile, an automatically produced profile plugs into the exact same pipeline and is evaluated by the same engine. This process is the subject of our current and future work; a first prototype generates candidate operators, checks them with a static and subprocess-based quality gate that executes the operator on the `OptDSL` model, and retains them only if they survive a holdout evaluation on seeds that were not used during generation. Automated operator development is not the focus of this technical report, which instead concentrates on the human-supervised LNS backend and its evaluation. In the present study this automated stage is left to subsequent work so that the human-supervised operators provide a controlled baseline.

Search loop. Figure 2 shows how the engine combines the general loop with the problem-specific profile. The construction operator builds a first feasible assignment, which is evaluated against the `OptDSL` model and becomes the current and the incumbent solution. Construction is profile-specific: depending on the class it either samples a random feasible assignment or runs a randomized constructive heuristic several times and keeps the best attempt. Each iteration then selects a destroy/repair pair, removes part of the current assignment, repairs it into a feasible candidate, and evaluates the candidate against the model's own constraints and objective. An acceptance criterion decides whether the candidate replaces the current solution: either a strict improving rule, or a simulated-annealing rule that also accepts worsening moves with a Metropolis probability and cools down over time. Independently of acceptance, the candidate replaces the incumbent whenever it improves on it. Operator selection is either uniform over

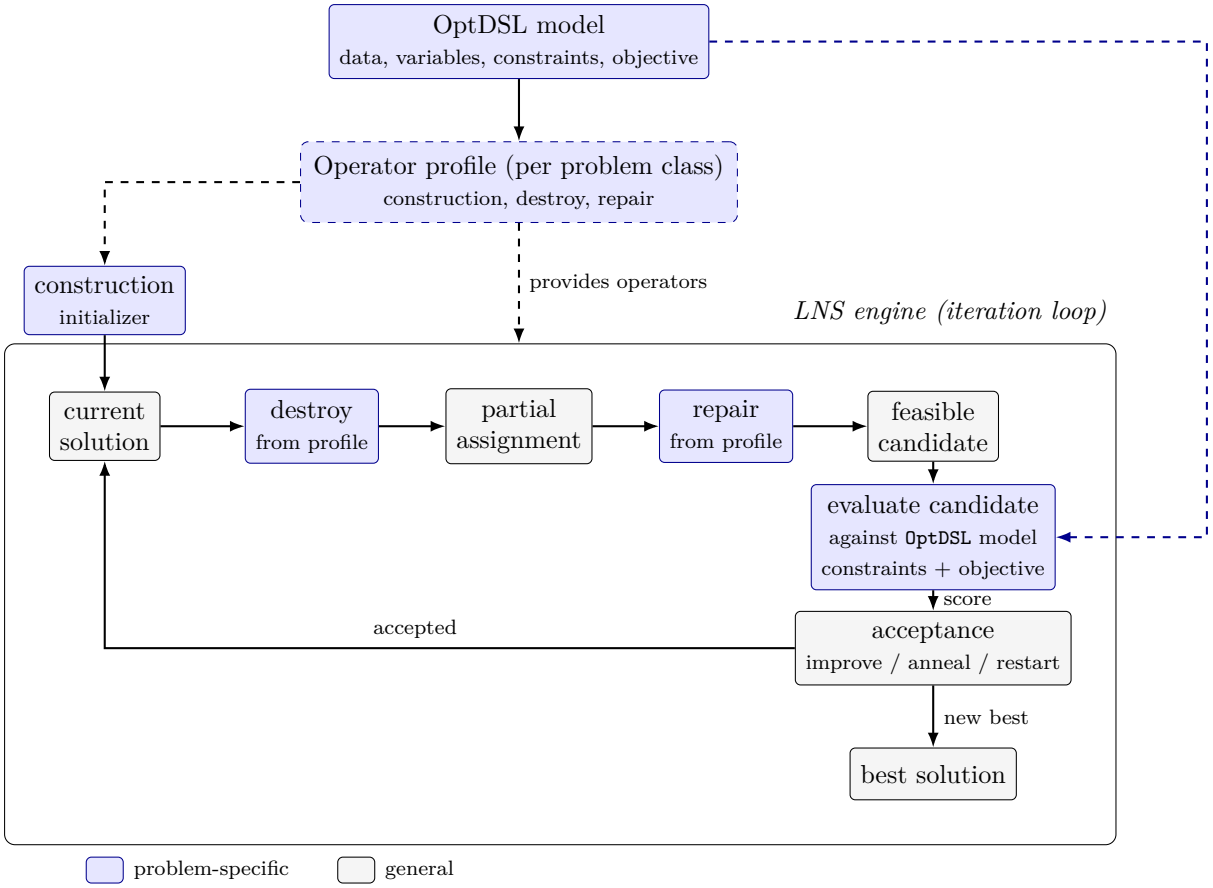


Figure 2: Design of the LNS backend with per-class operator profiles. Blue elements are problem-specific: the `OptDSL` model, the class profile that supplies the construction, destroy, and repair operators, and the evaluation, which runs against the `OptDSL` model itself. Grey elements form the general engine, which is independent of the problem class. The construction operator builds the initial solution; each iteration then destroys part of the current solution, repairs it into a feasible candidate, evaluates the candidate against the model's own constraints and objective, and feeds the outcome back into the loop.

the configured pairs or adaptive in the ALNS sense: each pair carries a weight that is updated from a decayed score, rewarding new best solutions, improvements over the current solution, and accepted worsening moves, and the destroy intensity is scaled up when the search stalls. Stagnation can additionally trigger a restart, which re-runs the construction operator and continues from the new assignment while keeping the incumbent. The loop stops after a fixed number of iterations, when a time limit is reached, or when no improvement has occurred for a given number of iterations.

4 Experimental Setup

The main objective of this research is to show that a problem formulated once in `OptDSL` can be used with two different backends, the exact MiniZinc-based backend and the LNS-based heuristic backend, and that both backends can work together on the same formulation. The experimental setup is therefore organized around this goal: we first describe the benchmark

families, then the baselines against which the two backends are compared.

The study currently uses small and medium-sized instances for the computational comparison. This restriction is deliberate: the aim is to demonstrate the feasibility of the unified workflow and the interaction of the two backends, not to establish a competitive benchmark result. Larger instances are the subject of ongoing work, where the binding costs are the `OptDSL`→MiniZinc translation on the exact side and the operator design on the heuristic side.

4.1 Benchmark Families

We evaluate on benchmark families from four problem classes, chosen for their complementary structure, their prevalence in the literature, and the availability of reference solutions.

Capacitated Vehicle Routing (CVRP). We use the classical Golden set [6, 10] and the E-n set of Christofides and Eilon [5], which are also distributed through the CVRPLIB [27]; see Toth and Vigo [26] for a comprehensive treatment.

Two-Dimensional Bin Packing (2DBP). We use the n/t instances of Hopper and Turton [11, 12], the Wang instances [28], and the instance classes introduced by Berkey and Wang [3]; exact methods and bounds follow Lodi et al. [15], Martello and Vigo [16]. In addition, we use an internally generated uniform-bin test set (`2D-BPP_CLASS`) that mirrors the Wang/Berkey–Wang class structure but is fully reproducible.

Job Shop Scheduling (JSSP). We use the standard families ta [25], la [14], abz [1], orb [2], and swv [23]. Reference makespans are taken from the published optima [2, 18, 25]; see also the survey of Jain and Meeran [13].

Woodcutting. The woodcutting class models a sawing line that cuts raw boards into pieces, filters out unusable pieces, optionally reorders boards and pieces, and stacks the remaining pieces into layers of beams of fixed length. All instances are internally generated with a problem instance generator and are stored together with the benchmark data. Their properties are listed in Table 2. Because these models are written as a pipeline of record-transforming machines rather than as a flat constraint set, they are reported separately in Section 5.2 rather than in Table 1.

4.2 Evaluation Baselines

The quality of the LNS backend is assessed against the exact MiniZinc-based backend on identical `OptDSL` instances. On each instance, the exact backend either proves optimality or, when the time limit is reached, yields a feasible incumbent together with an optimality gap from its reported bound; on some instances it finds no feasible solution at all, which we report as such. This provides two complementary baselines: reference (optimal) values for small instances, and a certified lower bound that limits the achievable gap of the heuristic on the larger benchmark families. Where published optima or best-known values exist, they serve as an additional independent reference [2, 16, 18, 25, 27].

5 Computational Study and Preliminary Results

The computational study compares the LNS-based heuristic backend against the MiniZinc-based exact backend on identical `OptDSL` instances under fixed iteration and time budgets. Only a selected subset of the available benchmark instances is used, restricted to small and medium-sized instances. This restriction is imposed by the current limitations of the `OptDSL`→MiniZinc translation, whose cost grows with the model size and becomes the binding factor on larger instances. Table 1 summarizes the outcome for the benchmark instances considered so far. All rows follow the same protocol: one `OptDSL` file per instance, a 120 s budget per backend, a single LNS seed, and the MiniZinc cp-sat backend with the `OptDSL`→MiniZinc translation included in the reported MiniZinc time. The *Size* column is class-specific (cities/vehicles/capacity for CVRP, jobs/machines for JSSP, bin dimensions/items for 2DBP). For each instance we report the objective value found by each backend, the bound reported by MiniZinc, the wall-clock time of each backend, and two relative gaps: the MiniZinc gap (MZ quality − MZ bound)/|MZ bound| and the LNS gap (LNS − MZ bound)/|MZ bound|.

5.1 Results on the Literature Benchmarks

Where known optima or best-known values are available, solution quality is benchmarked against them [2, 16, 18, 25, 27].

How to read the gap columns. The MiniZinc gap measures how far the exact backend's own incumbent is from its reported bound, and it is undefined (—) when MiniZinc found no feasible solution. The LNS gap is defined against the MiniZinc *bound*, not against the MiniZinc *quality*, so it carries two different meanings depending on whether MiniZinc closed the instance. When quality and bound coincide, the bound is a proof of optimality and the LNS gap is the true suboptimality of the LNS solution. When they differ, the LNS gap is dominated by the weakness of the dual bound and says nothing about how far the LNS solution is from the optimum.

This distinction matters for most rows: MiniZinc closed only `random_euclidean_cvrp_8`, `jssp_10x5`, `jssp_2x2`, `jssp_5x4`, `2d_bpp_inst_cl_06_020_10` and `2d_bpp_inst_cl_05_020_08`, whereas on `E-n13-k4`, all LA instances and `orb01` the reported bound lies far below the incumbent, and on the two random Euclidean CVRP instances and two 2DBP instances MiniZinc found no incumbent at all. The large gaps in the LA block (56–86 %) must therefore not be read as LNS being 56–86 % worse than the exact backend; they mainly reflect that the MiniZinc bound stays far below its own incumbent (e.g. 413 vs. 727 on `la01`).

Capacitated vehicle routing (CVRP). On the small instances, `random_euclidean_cvrp_8` is closed by both backends at the same objective value (370), with MiniZinc proving optimality in 2.2 s and LNS needing 1.8 s. On `E-n13-k4` the picture reverses: LNS reaches 247, the known optimum, in 3.6 s, whereas MiniZinc returns 279 with a bound of 139 and does not close the instance within its budget. On the two medium instances with 15 and 20 cities, MiniZinc reports a bound of 274 and 300 but no incumbent within its budget, while LNS returns 555 and 554.

Job shop scheduling (JSSP). On the small instances, both backends return the same

objective value on `jssp_2x2` (6) and `jssp_5x4` (22), with MiniZinc proving optimality in 0.8 s and 1.2 s respectively. On `jssp_10x5` MiniZinc proves the optimum of 37 and LNS returns 38, i.e. 2.7 % above the proven optimum. On the medium instances the exact backend closes none of the rows: on `la01`–`la05` and `orb01` the reported bound lies far below the incumbent, so the LNS gaps of 56–86 % must not be read as LNS being that much worse than the exact backend. The LA rows are limited by the exact backend rather than by the model: `chuffed` proves `la01`–`la05` optimal at 666, 655, 597, 590, and 593, i.e. exactly the published optima, whereas `cp-sat` returns incumbents 2–18 % above them within the same budget.

Two-dimensional bin packing (2DBP). On the small instances, `2d_bpp_inst_cl_06_020_10` is closed by both backends at objective 1, whereas on `2d_bpp_inst_cl_06_020_06` MiniZinc produced no feasible solution within its budget and LNS found a one-box packing in 69 s. On the medium instances, `2d_bpp_inst_cl_05_020_08` is closed by both backends at objective 5, whereas on `2d_bpp_inst_cl_04_020_08` MiniZinc found no feasible solution and on `2d_bpp_inst_cl_10_020_09` it returned a solution (6 boxes) but reported a bound of zero. These failures are a limitation of the *solver*, not of the model or the translation: on `2d_bpp_inst_cl_06_020_06` the same generated MiniZinc model is solved to proven optimality (objective 1) by both `gecode` and `chuffed`, whereas `cp-sat` returns UNKNOWN with zero solutions after 30 s. The disjunctive no-overlap encoding is therefore valid and solvable; it is the `cp-sat` backend that fails to find a feasible packing on these instances. The corresponding ∞ entries are consequently not evidence of LNS suboptimality; they mark a degenerate bound of zero. For the 2DBP class we therefore recommend `gecode` or `chuffed` over `cp-sat`. The 2DBP rows are limited to $n = 20$ because the `OptDSL`→MiniZinc translation becomes the binding cost beyond that size.

Runtime. LNS never exhausted its 120 s budget (maximum 69 s; every run terminated by the internal stop criterion), whereas MiniZinc used its full budget on the remaining twelve instances. The comparison is thus not wall-clock matched. On those twelve instances LNS is between about two and thirty-five times faster (`la01`: 9 s vs. 130 s; `E-n13-k4`: 3.6 s vs. 124 s), with the smallest speedups on two of the 2DBP instances (2.3× and 2.6×). On the tiny instances the two runtimes are comparable and the ranking is not stable (`jssp_5x4`: 1.9 s for LNS vs. 1.2 s for MiniZinc), so the remaining budget could be spent on additional iterations or restarts rather than being read as a speed advantage. Note that the reported MiniZinc time includes the `OptDSL`→MiniZinc translation, which dominates on the larger models: for `orb01` (10×10) the translation alone takes about 5 min, so the 471 s reported for that row are mostly translation rather than search.

Open issues. The study still uses a single seed and no independent feasibility check of the LNS output, so the table should be read as a first indication of relative behaviour rather than as a benchmark result.

5.2 Woodcutting

The woodcutting class is not part of Table 1 because its models are structurally different from the other three: instead of a flat constraint set over decision variables, they are written as a *pipeline* of "machines" that exchange whole record lists at module level, one machine per processing

Table 1: Comparison of MiniZinc-based exact solving and LNS-based heuristic search on identical `OptDSL` instances. Quality and time (in seconds) are as reported by each backend. The MiniZinc gap is (MZ quality − MZ bound)/|MZ bound| · 100 and measures how far the exact backend's own incumbent is from its reported bound. The LNS gap is (LNS − MZ bound)/|MZ bound| · 100 and is only interpretable as LNS suboptimality when MiniZinc closed the instance (quality = bound). *none* marks rows where MiniZinc found no feasible solution within its budget, — an undefined gap, and ∞ rows where the reported bound is zero. Bold marks the better of the two quality values in each row, and a bold bound marks a proven optimum (bound = quality). The 120 s budget limits the solver search; the reported MiniZinc time additionally includes the `OptDSL`→MiniZinc translation, which is why some rows exceed it.

Class	Set	Instance	Size	MZ Qual.	MZ Bound	MZ Gap	MZ Time	LNS Qual.	LNS Time	LNS Gap
CVRP	small	E-n13-k4	13 / 4 / 6000	279.0	139.0	100.72	123.9263	**247.0**	3.6291	77.70
CVRP	small	random_euclidean_cvrp_8	8 / 4 / 10	**370.0**	**370.0**	0.00	2.2398	**370.0**	1.8423	0.00
CVRP	medium	random_euclidean_cvrp_15	15 / 10 / 17	*none*	274.0	—	137.7895	**555.0**	10.1431	102.55
CVRP	medium	random_euclidean_cvrp_20	20 / 5 / 25	*none*	300.0	—	134.1788	**554.0**	9.6386	84.67
JSSP	small	jssp_10x5	10 / 5	**37.0**	**37.0**	0.00	56.5554	38.0	9.2919	2.70
JSSP	small	jssp_2x2	2 / 2	**6.0**	**6.0**	0.00	0.7777	**6.0**	0.4880	0.00
JSSP	small	jssp_5x4	5 / 4	**22.0**	**22.0**	0.00	1.2344	**22.0**	1.9066	0.00
JSSP	medium	la01	10 / 5	727.0	413.0	76.03	130.2423	**689.0**	9.4905	66.83
JSSP	medium	la02	10 / 5	**666.0**	394.0	69.04	129.5558	686.0	9.7288	74.11
JSSP	medium	la03	10 / 5	706.0	349.0	102.29	129.4788	**648.0**	9.5321	85.67
JSSP	medium	la04	10 / 5	625.0	369.0	69.38	129.4051	**621.0**	9.8149	68.29
JSSP	medium	la05	10 / 5	606.0	380.0	59.47	130.0443	**593.0**	9.5829	56.05
JSSP	medium	orb01	10 / 10	**1173.0**	695.0	68.78	470.5026	1198.0	47.7592	72.37
2DBP	small	2d_bpp_inst_cl_06_020_06.ins2D	300x300 / 20	*none*	0.0	—	157.3849	**1.0**	69.1330	∞
2DBP	small	2d_bpp_inst_cl_06_020_10.ins2D	300x300 / 20	**1.0**	**1.0**	0.00	97.8615	**1.0**	62.5340	0.00
2DBP	medium	2d_bpp_inst_cl_04_020_08.ins2D	100x100 / 20	*none*	0.0	—	152.3384	**1.0**	59.7394	∞
2DBP	medium	2d_bpp_inst_cl_05_020_08.ins2D	100x100 / 20	**5.0**	**5.0**	0.00	107.7868	**5.0**	13.3435	0.00
2DBP	medium	2d_bpp_inst_cl_10_020_09.ins2D	100x100 / 20	**6.0**	0.0	∞	158.2481	**6.0**	12.8442	∞

stage. We therefore document the class separately on its two smallest instances, which are small enough to reason about by hand.

Problem. A set of raw boards must be cut into pieces that are stacked into a given number of beams, each consisting of a fixed number of layers of fixed length. The boards carry defect intervals, so not every piece is usable, and the stacking imposes geometric restrictions on where a layer may be closed. The problem is to decide how to cut, select, and order the material so that the beams are filled exactly while wasting as little good material as possible.

The input fixes the geometry and the demand: the layer length `BEAM_LENGTH`, the number of layers per beam `BEAM_DEPTH`, the number of beams `N_BEAMS`, the unusable start and end margins `BEAM_SKIP_START` and `BEAM_SKIP_END`, the static forbidden zones `FORBIDDEN_INTERVALS`, and the minimum seam distance between consecutive layers `GAP_TO_BOARD_ABUT_IN_CONSECUTIVE_LAYERS`. It further fixes the board data (`N_BOARDS`, `MAX_BOARD_LENGTH`, the defect intervals per board) and the discretisation `MAX_N_CUTS_PER_BOARD`, from which the number of pieces `N_PIECES` follows. All instances studied here use the same geometry: `BEAM_LENGTH` $= 500$, `BEAM_SKIP_START` $=$ `BEAM_SKIP_END` $= 10$, `GAP_TO_BOARD_ABUT_IN_CONSECUTIVE_LAYERS` $= 10$, and the four forbidden zones $[90, 110]$, $[190, 210]$, $[290, 310]$ and $[390, 410]$; they differ in `N_BEAMS`, `BEAM_DEPTH`, the boards, and the defects.

The problem decomposes into five stages, each of which contributes decisions and constraints.

- *Board reordering* (optional) permutes the raw boards by adjacent swaps applied from left to right, so that the cutting stage sees them in a different order.
- *Cutting* splits every board at a bounded number of cut positions into pieces. The first cut is fixed at the start margin and the last at the board length minus the end margin, the cuts must be non-decreasing, and a piece is good if it overlaps no defect interval.
- *Filtering* decides for every piece whether it is kept or discarded. A bad piece can never be kept, and the length of every discarded good piece is charged to the objective.
- *Piece reordering* models a single-piece buffer.
- *Checking* validates the stream of kept pieces against the beam constraints: the running length of a layer never exceeds the layer length; every seam that does not close a layer lies strictly inside the usable part of the beam, outside the forbidden zones, and at least the required gap away from the seams of the previous layer of the same beam; and at the end exactly `N_BEAMS` $\times$ `BEAM_DEPTH` layers are completed with no partial layer left.

The objective is the total length of good material that is discarded. The input also declares `MAX_PIECES_PER_BEAM`, `MIN_DIST_BETWEEN_PIECES`, `MIN_LENGTH_OF_BOARD_IN_LAYER` and `MAX_SHIFT_CURVED_CUT`, but the current model body does not use them, so these rules are not enforced.

Instances. All instances are produced with a problem instance generator. It takes the beam geometry (layer length, margins, forbidden zones, seam gap), the number and length of the raw boards, and the defect rate as parameters, and writes one JSON file per instance. For a fixed

Table 2: Generated woodcutting instances. Boards gives the number and length of the raw boards, Beams × Layers the demand structure, Cuts the maximum number of cut positions per board (including both fixed ends), and Pieces = Boards × (Cuts −1). Defects counts defect intervals. Usable is the total board length minus both 10-unit margins, Demand is Beams × Layers ×500; for instances without defects their difference is a lower bound on the objective. The last column marks the instances studied in this report.

Instance	Boards	B × L	Cuts	Pieces	Defects	Usable	Demand	Studied
two_boards_two_layers_no_faults	2 (770, 270)	1 × 2	3	4	0	1000	1000	yes
one_beam_no_faults	4 (650)	1 × 5	4	12	0	2520	2500	yes
two_beams_no_faults	10 (650)	2 × 5	4	30	0	6300	5000	no
two_beams_02_bad_faults	20 (650)	2 × 5	4	60	23	12600	5000	no
10_beams_no_faults	45 (650)	10 × 5	4	135	0	28350	25000	no
10_beams_02_bad_faults	100 (650)	10 × 5	4	300	134	63000	25000	no
10_beams_05_bad_faults	200 (650)	10 × 5	4	600	835	126000	25000	no

random seed the generator is deterministic, so every instance is reproducible. Each generated file is converted to `OptDSL` by a converter script. Table 2 lists the generated instances. All of them share the beam geometry given above. The instance `two_boards_two_layers_no_faults` was designed as the smallest non-trivial case: its usable material equals the demand exactly, so the optimum wastes nothing (objective 0), yet that optimum can only be reached with a seam inside a layer, e.g. by cutting the 770 board into 500 + 250 and the 270 board into 250, giving the layers 500 and 250 + 250.

Experiments. The exact backend translates each `OptDSL` file to MiniZinc and solves the resulting model with MiniZinc 2.10.1 using chuffed, gecode and cp-sat (OR-Tools 9.15). The time limit is 60 s for `two_boards_two_layers_no_faults` and 120 s for `one_beam_no_faults`. Reported MiniZinc times are wall-clock times of the MiniZinc process (flattening and search); the `OptDSL`→MiniZinc translation is not included and takes 0.6–0.7 s and about 2 s, respectively. The heuristic backend runs the woodcutting LNS profile with 5000 iterations on the small instance and with 20 000 iterations on `one_beam_no_faults`. For the warm-start runs, LNS writes its incumbent as an `OptDSL` file with a `@warm_start` block (option `-emit-feasible-dsl`); the block carries the cut positions and the keep and reordering bits as solution hints and the incumbent's objective as an upper bound. We compare three variants: hints and bound, hints only, and bound only.

Results on the small instance. Table 3 shows that, with or without board reordering, chuffed, cp-sat and LNS all reach the optimum 0 in about one second; chuffed and cp-sat also prove optimality, and LNS finds the optimum as well. gecode finds no solution within 60 s from a cold start, even though the model has only 13 (respectively 14) decision variables. A warm start helps gecode only through the objective bound: gecode warns that it ignores the `warm_start` search annotation, so the hints-only run behaves like the cold start, whereas every run containing the bound proves the optimum in under half a second. This should not be overrated, because here the LNS bound already equals the optimum, so it leaves gecode only the task of finding a waste-free solution. cp-sat solves the instance from a cold start, so a warm start changes little. Board reordering does not change the outcome on this instance: no solver chooses to swap the boards, because the optimum is reachable without a swap.

Results on `one_beam_no_faults`. Four boards of length 650 must fill one beam of five

Table 3: Woodcutting instance `two_boards_two_layers_no_faults` without and with board reordering (60 s limit). opt. marks a proven optimum, unknown a run without any solution within the limit. Times are wall-clock seconds. LNS values are the best objective and the time range over repeated runs.

		Without board reordering		With board reordering	
Backend	Start	Objective	Time (s)	Objective	Time (s)
LNS	—	0	1.1–1.2	0	1.1–1.4
chuffed	cold	0 opt.	0.6	0 opt.	0.7
cp-sat	cold	0 opt.	0.6	0 opt.	0.8
cp-sat	hints + bound	0 opt.	0.5	0 opt.	0.3
cp-sat	hints only	0 opt.	0.3	0 opt.	0.4
cp-sat	bound only	0 opt.	0.4	0 opt.	0.4
gecode	cold	unknown	60.0	unknown	60.0
gecode	hints + bound	0 opt.	0.4	0 opt.	0.3
gecode	hints only	unknown	60.0	unknown	60.0
gecode	bound only	0 opt.	0.3	0 opt.	0.3

Table 4: Woodcutting instance `one_beam_no_faults` without board reordering (120 s limit). The volume lower bound on the objective is 20, so every solution with objective 20 is optimal. For warm starts, the time is the time to the first solution; none of the MiniZinc runs proves optimality within the limit. LNS uses 20 000 iterations.

Backend	Start	Objective	Time (s)	Status
LNS	—	20	5.0	optimal by the volume bound
chuffed	cold	20	118.3	feasible, no proof
cp-sat	cold	—	120.0	unknown, bound 0
gecode	cold	—	119.3	unknown
cp-sat	hints + bound	20	0.7	feasible, no proof
cp-sat	hints only	20	0.6	feasible, no proof
cp-sat	bound only	20	1.4	feasible, no proof
gecode	hints + bound	—	119.1	unknown
gecode	hints only	—	119.1	unknown
gecode	bound only	—	119.2	unknown

layers of 500. With 2520 usable units against a demand of 2500, at least 20 units must be wasted, so any solution with objective 20 is optimal. The model has 39 decision variables (16 cut positions, 12 keep bits and 11 piece-reordering bits). Table 4 summarizes the outcome. The LNS backend reaches the bound in about 5 s. Its solution cuts the four boards at $[10, 29, 529, 640]$, $[10, 399, 640, 640]$, $[10, 269, 640, 640]$ and $[10, 139, 639, 640]$, which yields pieces $19, 500, 111 \mid 389, 241, 0 \mid 259, 371, 0 \mid 129, 500, 1$. Discarding the 19-unit and 1-unit remainders leaves exactly 2500 units, which fill the five layers as 500, $111 + 389$, $241 + 259$, $371 + 129$ and 500; every layer is filled exactly, no seam falls into a forbidden zone, and the consecutive-layer seam distance is respected. From a cold start, chuffed also finds a solution of objective 20 within its budget but does not prove optimality. Its solution differs from the LNS one: it holds back the first piece with the single-piece buffer and releases it at the end of the stream, so both backends exercise the full pipeline including piece reordering. cp-sat and gecode find no solution from a cold start, and cp-sat reports a trivial bound of 0. With a warm start, cp-sat finds the optimum 20 within 0.6–1.4 s in all three variants, including hints only, but it still cannot prove optimality because its bound stays at 0, so it uses the full budget. gecode finds no solution in any warm-start variant: it ignores the hints, and the bound 20 alone does not propagate enough.

6 Conclusion

The study shows that a single problem formulation can be used across different optimization paradigms. The same `OptDSL` model is consumed by the exact MiniZinc backend and by the LNS backend, and both produce solutions of the same semantics on identical instances. The two paradigms differ in what they require and in what they deliver. The LNS backend needs an adaptation of the model: the search machinery, i.e. the construction, destroy, and repair operators, has to be provided per problem class, which is the price for a fast search that reaches good solutions under a fraction of the exact backend's budget. The exact backend, in contrast, is generic: it translates the model and hands it to a solver, which provides lower bounds and, where the search completes, proofs of optimality. The current limitation is on the exact side: the translation into MiniZinc produces large models that are hard to solve, so on the larger instances the solvers neither find good solutions nor prove optimality within the budget. The most immediate next step is therefore to automate the construction of the search machinery itself. Building on the observation that the LNS backend consumes the model directly, the construction, destroy, and repair operators can be generated from the model by a large language model and validated within the same workflow, so that switching paradigms eventually also automates the preparation of the heuristic search. This work is already in progress: a first prototype generates candidate operators from the model and retains them only if they pass an automated quality gate. The implementation, the problem instance generator, the converters, and the benchmark instances together with their converted `OptDSL` formulations are available at `https://github.com/RISCSoftware/inprossa`.

Acknowledgements

This work was funded by the Austrian Research Promotion Agency (FFG) within the project *InProSSA*—Industrial Problem Solving using Symbolic and Subsymbolic AI (grant number FO999923579).

References

[1] Joseph Adams, Egon Balas, and Daniel Zawack. The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34(3):391–401, 1988.

[2] David Applegate and William Cook. A computational study of the job-shop scheduling problem. *ORSA Journal on Computing*, 3(2):149–156, 1991.

[3] J. O. Berkey and Pearl Y. Wang. Two-dimensional finite bin-packing algorithms. *Journal of the Operational Research Society*, 38(5):423–429, 1987.

[4] Katharina Blaimschein. Autoformulation of optimization problems with trees of thoughts for llms. Master's thesis, Johannes Kepler University Linz, 2026.

[5] Nicos Christofides and Samuel Eilon. An algorithm for the vehicle-dispatching problem. *Operational Research Quarterly*, 20(3):309–318, 1969.

[6] Nicos Christofides, Aristide Mingozzi, and Paolo Toth. The vehicle routing problem. In Nicos Christofides, Aristide Mingozzi, Paolo Toth, and Claudio Sandi, editors, *Combinatorial Optimization*, pages 315–338. John Wiley & Sons, Chichester, UK, 1979.

[7] Emilie Danna, Edward Rothberg, and Claude Le Pape. Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming*, 102(1):71–90, 2005.

[8] Tereso del Río, Wolfgang Schreiner, Martina Seidl, Temur Kutsia, and Wolfgang Windsteiger. An intermediate representation format for industrial optimization problems—the translation of OptDSL to MiniZinc. RISC Report Series 26-04, Research Institute for Symbolic Computation (RISC), Johannes Kepler University Linz, 2026. ISSN 2791-4267. Available at `https://doi.org/10.35011/risc.26-04`.

[9] Matteo Fischetti and Andrea Lodi. Local branching. *Mathematical Programming*, 98(1–3): 23–47, 2003.

[10] Bruce L. Golden, Edward A. Wasil, James P. Kelly, and I-Ming Chao. The impact of metaheuristics on solving the vehicle routing problem: Algorithms, problem sets, and computational results. In Teodor Gabriel Crainic and Gilbert Laporte, editors, *Fleet Management and Logistics*, pages 33–56. Kluwer Academic Publishers, Boston, MA, 1998.

[11] Edwin Hopper and Brian C. H. Turton. A genetic algorithm for a 2D industrial packing problem. *Computers & Industrial Engineering*, 37(1–2):375–378, 1999.

[12] Eva Hopper and Brian C. H. Turton. An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem. *European Journal of Operational Research*, 128(1):34–57, 2001.

[13] A. S. Jain and S. Meeran. Deterministic job-shop scheduling: Past, present and future. *European Journal of Operational Research*, 113(2):390–434, 1999.

[14] Stephen Lawrence. Resource constrained project scheduling: An experimental investigation of heuristic scheduling techniques (update). Technical report, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, PA, 1984.

[15] Andrea Lodi, Silvano Martello, and Daniele Vigo. Recent advances on two-dimensional bin packing problems. *Discrete Applied Mathematics*, 123(1–3):379–396, 2002.

[16] Silvano Martello and Daniele Vigo. Exact solution of the two-dimensional finite bin packing problem. *Management Science*, 44(3):388–399, 1998.

[17] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. Minizinc: Towards a standard CP modelling language. In *Principles and Practice of Constraint Programming (CP 2007)*, volume 4741 of *LNCS*, pages 529–543. Springer, 2007.

[18] Eugeniusz Nowicki and Czesław Smutnicki. A fast taboo search algorithm for the job shop problem. *Management Science*, 42(6):797–813, 1996.

[19] David Pisinger and Stefan Ropke. Large neighborhood search. In *Handbook of Metaheuristics*, volume 146 of *International Series in Operations Research & Management Science*, pages 399–419. Springer, 2 edition, 2010.

[20] David Pisinger and Stefan Røpke. Large neighborhood search. In Michel Gendreau and Jean-Yves Potvin, editors, *Handbook of Metaheuristics*, volume 272 of *International Series in Operations Research & Management Science*, pages 99–127. Springer, 3 edition, 2019.

[21] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.

[22] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming (CP 1998)*, volume 1520 of *LNCS*, pages 417–431. Springer, 1998.

[23] Robert H. Storer, S. David Wu, and Renzo Vaccari. New search spaces for sequencing problems with application to job shop scheduling. *Management Science*, 38(10):1495–1509, 1992.

[24] Peter J. Stuckey, Thibaut Feydy, Andreas Schutt, Guido Tack, and Julien Fischer. The MiniZinc challenge 2008–2013. *AI Magazine*, 35(2):55–60, 2014.

[25] Éric D. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.

[26] Paolo Toth and Daniele Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications.* SIAM, 2 edition, 2014.

[27] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Anand Subramanian, and Thibaut Vidal. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.

[28] Pearl Y. Wang. Two algorithms for constrained two-dimensional cutting stock problems. *Operations Research*, 31(3):573–586, 1983.